CS6230 - Assignment 1

Yasho Srikar: EE23B114

1 Microarchitecture and Algorithm

The BSV floating-point adder is parameterized by exponent width e and fraction width m, with legal ranges

$$2 \leq e \leq 15, \qquad 2 \leq m \leq 63.$$

The synthesized DUT uses IEEE binary32 parameters,

$$e = 8, \qquad m = 23.$$

The operands are first unpacked and classified into zero, subnormal, normal, infinity, quiet NaN, and signalling NaN cases. Special cases are resolved before ordinary arithmetic.

For finite non-zero operands, the magnitudes of the operands are compared and the operand with the larger magnitude is selected. The smaller significand is right shifted according to the exponent difference while retaining guard, round, and sticky information.

For operands with equal signs, the significands are added. For opposite signs, magnitude subtraction is performed. Exact cancellation produces positive zero.

The result is then normalized and rounded using round-to-nearest, ties-to-even. The rounding decision is based on the guard bit, the sticky information, and the least-significant retained bit:

$$\text{roundUp} = G \wedge (S \vee L).$$

Overflow produces infinity and sets the corresponding exception flags. Underflow is detected after rounding for tiny inexact results.

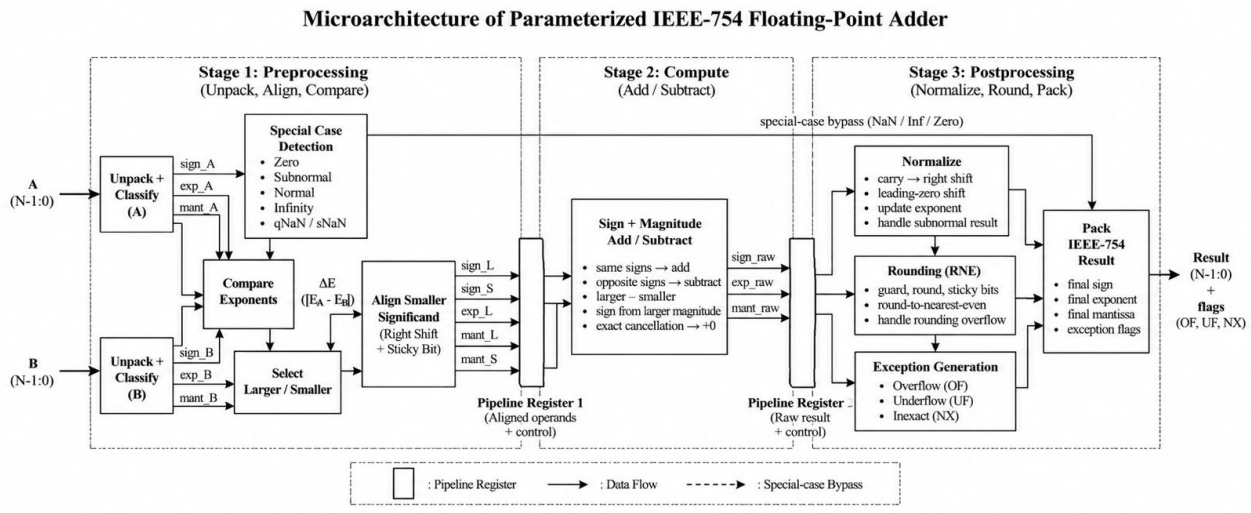


Figure 1: Floating-Point Adder Microarchitecture

2 Pipeline and Latency

The floating-point adder is organized into three main computation stages with FIFO/register boundaries between them. The first stage, implemented by the `preAlign` rule, performs operand classification, special-case detection, exponent comparison, and significand alignment. The aligned operands are stored in `stage1Q`.

The second stage, implemented by the `addSubtract` rule, performs the actual significand addition or subtraction. The larger magnitude operand is selected during alignment, and the sign of the operation determines whether the significands are added or subtracted. The resulting magnitude, sign, and exponent are stored in `normReg`.

The final stage consists of normalization and result packing. For subtraction, the `normalizeStep` rule shifts the result left until the leading significand bit is restored or the minimum exponent is reached. The `roundPack` rule then performs round-to-nearest, ties-to-even rounding, handles rounding overflow, packs the sign, exponent, and mantissa fields, and generates the exception flags.

The main operations performed across the stages are:

- Classification of zero, subnormal, normal, infinity, and NaN operands.
- Comparison of operand magnitudes and exponent alignment.
- Generation of guard, round, and sticky information during right shifting.
- Significand addition or magnitude subtraction.
- Cancellation normalization for subtraction results.
- Round-to-nearest, ties-to-even.
- Overflow, underflow, and inexact exception generation.
- Final floating-point result packing.

The design uses FIFOs to decouple the input, alignment, arithmetic, and output portions of the datapath. This allows transactions to be buffered between computation stages while preserving the ordering of results. The normalization logic is implemented using a variable left shift rather than an iterative one-bit-at-a-time datapath, reducing the number of cycles required for cancellation cases.

3 Difficult Corner Cases

The difficult cases encountered during implementation were primarily associated with special values, subnormal numbers, cancellation, and rounding.

Subnormal operands do not contain an implicit hidden one and require different effective exponent handling from normal operands. Exact cancellation must produce +0.

Large exponent differences require sticky-bit generation because bits shifted out of the retained significand can affect the rounding decision.

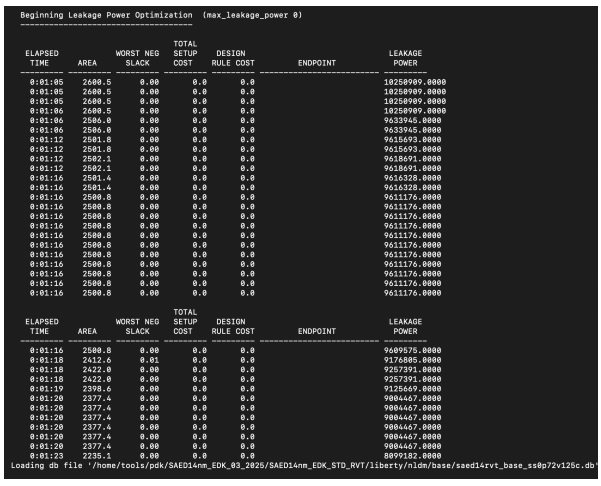
Halfway cases require ties-to-even rounding. Therefore, the retained least-significant bit must be considered when deciding whether to increment the significand.

Cancellation close to the minimum normal exponent also requires careful normalization so that a valid subnormal result is produced instead of prematurely flushing the result to zero.

NaN and infinity cases require priority handling. In particular, signalling NaNs raise the invalid exception, while the addition of opposite-signed infinities produces a NaN with the invalid flag set.

3.1 Issues faced

Tried to change tcl file for power report but couldn't get that done. I added the tcl log for power instead.



```
Beginning Leakage Power Optimization  (max_leakage_power 0)
-----------------------------------------

                                  TOTAL
ELAPSED            WORST NEG      SETUP    DESIGN                                LEAKAGE
 TIME      AREA      SLACK        COST    RULE COST        ENDPOINT             POWER
-------  --------  ---------  ----------  ---------  ------------------------  ----------
 0:01:05    2600.5       0.00         0.0        0.0                            10258909.0000
 0:01:05    2600.5       0.00         0.0        0.0                            10258909.0000
 0:01:05    2600.5       0.00         0.0        0.0                            10258909.0000
 0:01:06    2600.5       0.00         0.0        0.0                            10258909.0000
 0:01:06    2586.0       0.00         0.0        0.0                            9633945.0000
 0:01:06    2586.0       0.00         0.0        0.0                            9633945.0000
 0:01:12    2581.8       0.00         0.0        0.0                            9615693.0000
 0:01:12    2581.8       0.00         0.0        0.0                            9615693.0000
 0:01:12    2582.1       0.00         0.0        0.0                            9618691.0000
 0:01:12    2582.1       0.00         0.0        0.0                            9618691.0000
 0:01:16    2581.4       0.00         0.0        0.0                            9616328.0000
 0:01:16    2581.4       0.00         0.0        0.0                            9616328.0000
 0:01:16    2580.8       0.00         0.0        0.0                            9611176.0000
 0:01:16    2580.8       0.00         0.0        0.0                            9611176.0000
 0:01:16    2580.8       0.00         0.0        0.0                            9611176.0000
 0:01:16    2580.8       0.00         0.0        0.0                            9611176.0000
 0:01:16    2580.8       0.00         0.0        0.0                            9611176.0000
 0:01:16    2580.8       0.00         0.0        0.0                            9611176.0000
 0:01:16    2580.8       0.00         0.0        0.0                            9611176.0000
 0:01:16    2580.8       0.00         0.0        0.0                            9611176.0000
 0:01:16    2580.8       0.00         0.0        0.0                            9611176.0000

                                  TOTAL
ELAPSED            WORST NEG      SETUP    DESIGN                                LEAKAGE
 TIME      AREA      SLACK        COST    RULE COST        ENDPOINT             POWER
-------  --------  ---------  ----------  ---------  ------------------------  ----------
 0:01:16    2580.8       0.00         0.0        0.0                            9609575.0000
 0:01:18    2412.6       0.01         0.0        0.0                            9176805.0000
 0:01:18    2422.0       0.00         0.0        0.0                            9257391.0000
 0:01:18    2422.0       0.00         0.0        0.0                            9257391.0000
 0:01:19    2398.6       0.00         0.0        0.0                            9125669.0000
 0:01:20    2377.4       0.00         0.0        0.0                            9004467.0000
 0:01:20    2377.4       0.00         0.0        0.0                            9004467.0000
 0:01:20    2377.4       0.00         0.0        0.0                            9004467.0000
 0:01:20    2377.4       0.00         0.0        0.0                            9004467.0000
 0:01:20    2377.4       0.00         0.0        0.0                            9004467.0000
 0:01:20    2377.4       0.00         0.0        0.0                            9004467.0000
 0:01:23    2235.1       0.00         0.0        0.0                            8099182.0000
Loading db file '/home/tools/pdk/SAED14nm_EDK_03_2025/SAED14nm_EDK_STD_RVT/liberty/nldm/base/saed14rvt_base_ss0p72v125c.db'
```

Figure 2: Power log details

Also for 0.7 ns clock period I am facing 4 path violations with slack of -00 which i think is precision issue. So I increased it to 0.71 ns.

4 Synthesis Results and Critical Path

The floating-point adder was synthesized using the SAED14nm standard cell library at the $ss0p72v125c$ operating corner. The clock period was set to 0.71 ns, corresponding to a clock frequency of approximately 1.408 GHz.

The important synthesis metrics obtained from Design Compiler are summarized below.

Table 1: Synthesis Results

Metric	**Result**
Worst register-to-register path	0.69 ns
Clock period	0.71 ns
Corresponding frequency	1.408 GHz
Worst setup slack	0.00 ns
Total negative slack	0.00 ns
Leaf cells	3369
Combinational cells	2900
Sequential cells	469
Cell area	2235.10 μm^2
Design area including nets	4007.97 μm^2
Dynamic power	TBD
Leakage power	TBD
Total power	TBD

The worst register-to-register path belongs to the `f2f` path group. The critical path starts at

`adder_inputQ/data0_reg_reg_33_`

and ends at

`adder_stage1Q/data0_reg_reg_6_`.

The path has a critical delay of approximately

$$T_{\text{critical}} = 0.69 \text{ ns}.$$

With a clock period of 0.71 ns, the corresponding clock frequency is

$$f_{\text{clk}} = \frac{1}{0.71 \text{ ns}} \approx 1.408 \text{ GHz}.$$

The critical path contains 29 levels of logic. The timing report shows that the f2f path is the limiting path, with approximately zero setup slack at the selected clock period.

The other major timing groups have additional margin. The register-to- output path has a reported slack of 0.34 ns, while the input-to-register path has approximately 0.00 ns slack.

The synthesis report contains no setup or hold violations at the reported precision. The overall design WNS and TNS are both 0.00 ns.

5 Functional Validation and Limitations

The design was functionally validated using automatically generated floating-point test vectors. The testbench compares the DUT output against the corresponding expected hexadecimal result.

Testing was performed for the following (e, m) configurations:

Table 2: Functional Validation Configurations

Configuration	e	m
OCP FP8 E4M3	4	3
Binary16	5	10
Bfloat16	8	7
Binary32	8	23
Binary64	11	52
Extra narrow	2	2
Extra wide	15	63

The final functional validation was automated using the testbench and the vector-generation script. The results for each configuration are summarized below.

Table 3: Functional Test Results

Configuration	e	m	**Vectors**	**Result**
OCP FP8 E4M3	4	3	20000	PASS
Binary16	5	10	20000	PASS
Bfloat16	8	7	20000	PASS
Binary32	8	23	20000	PASS
Binary64	11	52	20000	PASS
Extra narrow	2	2	20000	PASS
Extra wide	15	63	20000	PASS

The implementation currently supports round-to-nearest, ties-to-even.

The functional testing covered multiple exponent and significand widths, including standard binary floating-point formats and the additional narrow and wide parameter configurations.

The synthesis results reported in this section apply specifically to the binary32 configuration ($e = 8, m = 23$).